



Algorithms and Data Structures in C

Practical II - "Equations"

Deadline: $\begin{cases} \text{Monday, February 16, 2026, at 9am, for part 1} \\ \text{Monday, February 23, 2026, at 9am, for part 2 + bonuses} \end{cases}$

In this assignment, we will use linked lists to recognize and solve equations.

Not all equations are easily solvable; therefore, we restrict ourselves to linear equations, such as $3x + 4 = 2 - x$, $-8x + 1 = 0$ and $x = 3x + 1$. For further simplification, we require that the equations do not contain parentheses, so e.g. $4(x + 1) = 3$ is excluded. Moreover, we only solve linear equations in one variable, and not e.g. $x + y + 3 = x - 5$.

The assignment has two parts and both should be submitted on Themis.

Part 1: recognizing and classifying the equation

First, you have to write a recognizer that recognizes equations.

Grammar

The equations to be recognized in this assignment are generated by the following grammar:

```
 $\langle \text{equation} \rangle ::= \langle \text{expression} \rangle \text{'='} \langle \text{expression} \rangle$   
 $\langle \text{expression} \rangle ::= [ \text{'-'} ] \langle \text{term} \rangle \{ \text{'+'} \langle \text{term} \rangle \mid \text{'-'} \langle \text{term} \rangle \}$   
 $\langle \text{term} \rangle ::= \langle \text{nat} \rangle \mid [ \langle \text{nat} \rangle ] \langle \text{identifier} \rangle [ \text{'^'} \langle \text{nat} \rangle ]$   
 $\langle \text{nat} \rangle ::= \langle \text{digit} \rangle \{ \langle \text{digit} \rangle \}$   
 $\langle \text{digit} \rangle ::= \text{'0'} \mid \text{'1'} \mid \text{'2'} \mid \dots \mid \text{'8'} \mid \text{'9'}$   
 $\langle \text{identifier} \rangle ::= \langle \text{letter} \rangle \{ \langle \text{digit} \rangle \mid \langle \text{letter} \rangle \}$   
 $\langle \text{letter} \rangle ::= \text{'a'} \mid \text{'b'} \mid \dots \mid \text{'z'} \mid \text{'A'} \mid \text{'B'} \mid \dots \mid \text{'Z'}$ 
```

You may use the functions described in Sections 1.4.4 (Scanning) and 1.4.5 (Recognition) of the *Lecture notes*. These are available on Themis.



Alert!

Here we only want a special kind of expressions, defined by the line for $\langle \text{expression} \rangle$ above. This differs from the expressions used in Section 1.4 of the *Lecture notes* and by the function `acceptExpression` in the file `recognizeExp.c`.



Note.

You should copy `recognizeExp.c` to `recognizeEq.c` and adapt it to the grammar above. Hint: the functions `acceptExpression` and `acceptTerm` should be changed, but the functions `acceptNumber`, `acceptIdentifier` and `acceptCharacter` should not.



Your task

Your program should not solve all equations generated by this grammar. In this first part, your program should check

- whether the input is an equation, i.e. a production of $\langle equation \rangle$;
- if so, whether it is an equation in one variable;
- if so, what its degree is.

The first test case on Themis contains some edge cases involving the number 0, so it clear how these should be handled. The bottom line: treat 0 like any other number (and don't cancel it out).

The communication with your program should run via a dialogue, as with the evaluation of arithmetical expressions in Sections 1.4.5 and 1.4.6 of the *Lecture notes*.

Example input and output

Input:

```
3 + x - 3x = - x^2 - 3x^3
2x^1 - 4x^0 = 7
x + y = 0
2 longvariablename = 6

x^2 + 5x - 7
2x = 3 = 4y
= 3x + 7
x^2 + x == 3
x (x + 3) = 7
!
```

Output:

```
give an equation: 3 + x - 3 x = - x ^ 2 - 3 x ^ 3
this is an equation in 1 variable of degree 3

give an equation: 2 x ^ 1 - 4 x ^ 0 = 7
this is an equation in 1 variable of degree 1

give an equation: x + y = 0
this is an equation, but not in 1 variable

give an equation: 2 longvariablename = 6
this is an equation in 1 variable of degree 1

give an equation:
this is not an equation

give an equation: x ^ 2 + 5 x - 7
this is not an equation

give an equation: 2 x = 3 = 4 y
this is not an equation

give an equation: = 3 x + 7
this is not an equation

give an equation: x ^ 2 + x = = 3
this is not an equation

give an equation: x ( x + 3 ) = 7
this is not an equation

give an equation: good bye
```

Themis

The files `scanner.c` and `scanner.h` will be inserted by Themis automatically — you do not need to re-upload them, and you should not have any conflicting definitions in your own files.

You could, for example, structure your solution into three files: `recognizeEq.h` (based on `recognizeExp.h`), `recognizeEq.c` (based on `recognizeExp.c`), and `mainRecog.c`.

Part 2: solving the equations

Now extend the program for Part 1 by giving the solution in the case that it is an equation of one variable of degree 1. Print the answer, rounded to 3 decimals. When the equation is not solvable (i.e. there are no solutions, or infinitely many), you print: **not solvable**.



Note.

When applied to two expressions of type `int`, the C operator `'/'` performs *integer division*, e.g. `1 / 3 == 0`. This is also the case when using two integer variables, e.g.:

```
int a = 1;
int b = 3;
double c = a / b;      // c == 0.0
```

To do a floating point division, you should cast one of the operands (or both) into a floating point number type. For example, you can write `'a / (double) b'`. See the lecture notes of *Programming Fundamentals (Imperative Programming)*, Section 1.6.1.



Note.

When printing real numbers with e.g. 3 decimals, C will print `-0.000` for numbers between `-0.0005` and `0`. To prevent this, convert these values to `0` before printing.



Note.

We define 0^0 to be equal to 1. This allows you to simplify terms like $3x^0$ to 3.



Example input and output

Input:

```
x - 34 = - 15x + 14
4y + 1 = y
10000z = -1
x^2 = 4
5x - 7
2x = 3 = 4y
= 3x + 7
!
```

Output:

```
give an equation: x - 34 = - 15 x + 14
this is an equation in 1 variable of degree 1
solution: 3.000

give an equation: 4 y + 1 = y
this is an equation in 1 variable of degree 1
solution: -0.333

give an equation: 10000 z = - 1
this is an equation in 1 variable of degree 1
solution: 0.000

give an equation: x ^ 2 = 4
this is an equation in 1 variable of degree 2

give an equation: 5 x - 7
this is not an equation

give an equation: 2 x = 3 = 4 y
this is not an equation

give an equation: = 3 x + 7
this is not an equation

give an equation: good bye
```

Themis

The same two files as for Part 1 (`scanner.c`, `scanner.h`) will be automatically included by Themis.



Bonus assignments

- Extend your solver to equations of degree 2. If there are two solutions, the greater one should be printed first. Example input and output:

Input:

```
x^2 = 4
1 + x - 3x = - x^2
2 long ^ 2 - 4long^1 = 7
10 x^2 - 10 = -10
two^2 - 2 = 2 =
y^2 + x^2 = 5 x
25y + 12 = y
!
```

Output:

```
give an equation: x ^ 2 = 4
this is an equation in 1 variable of degree 2
solution: 2.000 and -2.000

give an equation: 1 + x - 3 x = - x ^ 2
this is an equation in 1 variable of degree 2
solution: 1.000

give an equation: 2 long ^ 2 - 4 long ^ 1 = 7
this is an equation in 1 variable of degree 2
solution: 3.121 and -1.121

give an equation: 10 x ^ 2 - 10 = - 10
this is an equation in 1 variable of degree 2
solution: 0.000

give an equation: two ^ 2 - 2 = 2 =
this is not an equation

give an equation: y ^ 2 + x ^ 2 = 5 x
this is an equation, but not in 1 variable

give an equation: 25 y + 12 = y
this is an equation in 1 variable of degree 1
solution: -0.500

give an equation: good bye
```

- Extend your recognizer and solver to pairs of linear equations with two variables:

Input:

```
3x + y = 4
6y = 4 - 4x
27=2apples+14bananas-11
- apples = 3 bananas - 11
x = 10
a + x = 100
7a + 7b -5a = 5a - 3a
23b - 5a = 22a - 27a
!
```

Output:

```
give an equation: 3 x + y = 4
give an equation: 6 y = 4 - 4 x
solution: x = 1.429 and y = -0.286

give an equation: 27 = 2 apples + 14 bananas - 11
give an equation: - apples = 3 bananas - 11
solution: apples = 5.000 and bananas = 2.000

give an equation: x = 10
give an equation: a + x = 100
solution: x = 10.000 and a = 90.000

give an equation: 7 a + 7 b - 5 a = 5 a - 3 a
give an equation: 23 b - 5 a = 22 a - 27 a
not solvable

give an equation: good bye
```



Grading

Parts 1 and 2 are worth $4 + 4$ points (equally distributed over all test cases). Then, 2 points are awarded for code quality: *for this, we only consider your submission to Part 2*. The bonus assignments are worth $1 + 1$ points (equally distributed across all test cases). This means that in total, you can obtain 12 points. However, grades above 10 are capped at 10.

As always, there is no submission limit and Themis considers your latest submission.

For this assignment, as an exception to the rule, we have made all test cases visible and downloadable on Themis. You are free to use them to debug your program.